

II_Project — Architecture Documentation

Bengos Restaurant Management System Module Architecture Specifications

1. Architecture Patterns — How They Work

1.1 Code-Behind Pattern

Definition: The code-behind pattern is the default model for WPF and WinForms applications. The UI is defined in a markup/designer file (.xaml or .Designer.cs), and all logic — event handlers, data access, business rules — lives in a companion "code-behind" file (.xaml.cs or .cs).

How it works:

- The XAML/Designer file declares UI elements (buttons, grids, textboxes).
- The code-behind file references these elements by name (e.g., txtUsername, btnLogin).
- Event handlers (Click, TextChanged, SelectionChanged) are wired to methods in the code-behind.
- All logic executes directly in these handlers — validation, database queries, calculations.
- There is no intermediate layer between the UI and the logic.

Flow Diagram:



Used in:

- BengosRestaurantApp, Staff_Management, Billing&Payments, Inventory

1.2 MVC (Model-View-Controller)

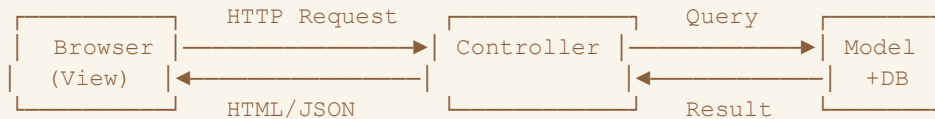
Definition: MVC separates an application into three interconnected components: the Model (data + business rules), the View (UI), and the Controller (handles input, updates Model, selects View).

How it works:

- Controller receives an HTTP request (e.g., GET /Menu/Category/Main).
- Controller interacts with the Model (via DbContext / repositories) to fetch or mutate data.

- Controller selects a View and passes the Model data to it (via ViewBag or a strongly-typed model).
- View renders HTML using Razor syntax, displaying the data from the Model.
- User interaction (click, form submit) sends a new HTTP request to the Controller — cycle repeats.

Flow Diagram:



Key characteristics:

- Separation of concerns: presentation (Razor Views), logic (Controllers), data (Models).
- Each Controller action is a concise method that typically does 3 things: get data, process, return view.
- Dependency Injection is used (e.g., RestaurantContext injected into MenuController).

Used in:

- DigitalClientMenu (BengosMenu) — the only module currently using a formal pattern.

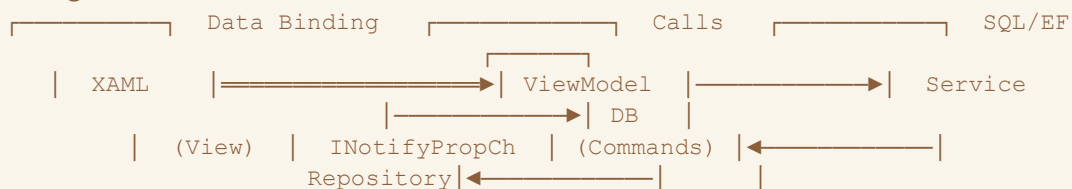
1.3 MVVM (Model-View-ViewModel)

Definition: MVVM is the recommended architecture for WPF applications. It separates the UI (View) from the presentation logic (ViewModel) and the data (Model), using data binding and commands as the communication mechanism.

How it works:

- View (XAML) declares the UI and binds to properties and commands on the ViewModel.
- ViewModel exposes ObservableCollection properties, ICommand objects, and INotifyPropertyChanged events.
- View binds to ViewModel via DataContext (e.g., <Window DataContext="{Binding MainViewModel}">).
- When the user interacts with the UI, the binding engine automatically calls the corresponding command or updates the bound property.
- ViewModel calls Service/Repository methods to get/save data from the Model layer.
- Any change to a ViewModel property automatically updates the UI via data binding (two-way).

Flow Diagram:





Key characteristics:

- View knows about ViewModel; ViewModel knows about Model; Model knows nothing about the others.
- Two-way data binding: UI changes the ViewModel and ViewModel changes update the UI automatically.
- Testable: ViewModel logic can be unit-tested without the UI.
- Commands (ICommand) replace Click event handlers.

Recommended for:

- BengosRestaurantApp (WPF) — replace code-behind with ViewModels and data binding.
- inventoryWPF (WPF) — already has INotifyPropertyChanged on Product, just needs a ViewModel layer.

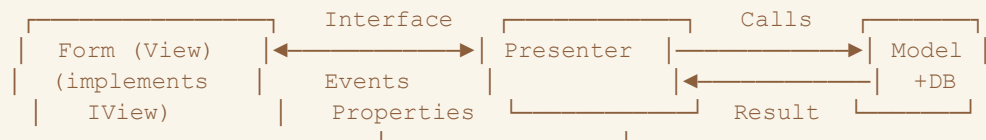
1.4 MVP (Model-View-Presenter)

Definition: MVP is the recommended architecture for WinForms applications. The View (Form) is passive and delegates all user actions to a Presenter, which contains the presentation logic and updates the View through an interface.

How it works:

- View (Form) implements an IView interface with properties and events.
- Presenter receives a reference to the IView interface (not the concrete Form).
- View raises events (button click, selection changed) → Presenter handles them.
- Presenter calls Model / Service layer to get data.
- Presenter updates the View through the IView interface (sets properties like StaffName, ShiftList).
- View simply displays whatever the Presenter gives it — it contains no logic.

Flow Diagram:



Key characteristics:

- View is passive: it only displays data and forwards user actions.
- Presenter contains all presentation logic and is fully unit-testable.
- View-Presenter communication is through a defined interface (loose coupling).
- Better suited than MVVM for WinForms since WinForms lacks native data binding.

Recommended for:

- Staff_Management (WinForms) — extract logic from MainForm.cs into a Presenter.

- Billing&Payments (WinForms) — extract billing logic from Form1.cs into a Presenter.

1.5 Architecture Comparison

Aspect	Code-Behind	MVC	MVVM	MVP
Platform	WPF / WinForms	ASP.NET Core (Web)	WPF	WinForms
Separation	None	Controller↔Model↔View	View↔VM↔Model	View↔Presenter↔Model
UI Updates	Direct (code)	HTTP Response (new page)	Data Binding (auto)	Via Interface (manual)
Testability	Low	Medium	High	High

2. Detailed Module Specifications

Module 1: BengosRestaurantApp (Main Hub)

Project Path	BengosRestaurantApp/
Technology	WPF (.NET 8.0-windows) — WinExe
Current Architecture	Code-Behind (no formal pattern)
Recommended Architecture	MVVM (Model-View-ViewModel)
Description	Central dashboard application that launches all other restaurant management modules. Contains 7 windows: MainWindow (hub with navigation buttons), LoginWindow (user authentication from users.txt), StaffWindow (shift CRUD with role-based permissions), InventoryWindow (in-memory product CRUD), BillingWindow (order entry + discount), PaymentWindow (cash/card payment), MenuWindow (local menu browser with category pills).
Data Access	Text files (users.txt, shifts.txt) via System.IO; in-memory ObservableCollection for orders and inventory.
Current Problem	All logic — event handling, validation, file I/O, and business rules — lives in .xaml.cs code-behind files. No separation of concerns, hard to test, hard to maintain.
MVVM Refactor	Extract ViewModels (MainViewModel, LoginViewModel, StaffViewModel, etc.) with ObservableProperties and ICommands. Move file I/O to a Repository layer. Bind XAML to ViewModel via DataContext.

Module 2: DigitalClientMenu (Web Digital Menu)

Project Path	DigitalClientMenu/BengosMenu/
Technology	ASP.NET Core MVC (.NET 10.0) — Web Application
Current Architecture	MVC (Model-View-Controller) ✓
Recommended Architecture	MVC (keep — already correct)
Description	Public-facing web application for customers to browse the restaurant menu. HomeController serves the landing page; MenuController serves category-filtered dish listings with ingredient details. Data is seeded on first run via Program.cs.
Data Access	Entity Framework Core 10.0 + SQLite (restaurant.db). RestaurantContext with DbSet<Dish>, DbSet<Produs>, DbSet<DishIngredient>.
Current Strength	Clean separation: Controllers handle HTTP, Models are entities, Data layer has DbContext, Views render HTML. DI for DbContext. This is the model module in the project.
Note	No changes needed architecturally. Consider adding a Service layer between Controllers and DbContext for thicker business logic.

Module 3: Staff_Management (Shift Scheduling)

Project Path	Staff_Management/hw1/
Technology	WinForms (.NET 8.0-windows) — WinExe
Current Architecture	Code-Behind (no formal pattern)
Recommended Architecture	MVP (Model-View-Presenter)
Description	Staff shift scheduling application. LoginForm authenticates users from users.txt (format: username,password,role). MainForm provides CRUD for shifts with role-based permissions: Admin can manage all staff shifts; regular staff can only manage their own. Shift data includes staff member, day, shift type (morning/evening/night), and overtime flag.
Data Access	Text files (users.txt, shifts.txt) via System.IO.
Current Problem	MainForm.cs is 583 lines — UI creation (manual), event handlers, file I/O, validation, and permissions logic are all in one class. WinForms designer file is not used (controls built in code).
MVP Refactor	Create IShiftView interface (properties: StaffList, DayList, ShiftTypeList, SelectedShift, etc. + events: AddClicked, DeleteClicked). Create

	ShiftPresenter (handles events, calls ShiftRepository, updates View via interface). Keep Form thin — only UI setup and interface implementation.
--	--

Module 4: Billing & Payments

Project Path	Billing&Payments/Homework/
Technology	WinForms (.NET 8.0-windows) — WinExe
Current Architecture	Code-Behind (no formal pattern)
Recommended Architecture	MVP (Model-View-Presenter)
Description	Order billing and payment processing. Form1 displays an order summary (hardcoded sample items: Cappuccino, Tiramisu, IceCream) with editable quantities, discount calculation popup, subtotal/discount/total display. Form2 handles payment — cash (with change calculation) or card.
Data Access	In-memory (List<OrderItem>). No persistence.
Current Problem	Order management, discount logic, and payment validation are all in Form1.cs and Form2.cs. Cannot unit-test billing rules without opening the Form.
MVP Refactor	Create IBillingView interface and BillingPresenter. Move discount calculation and order total logic to Presenter. Create IPaymentView and PaymentPresenter for payment processing. Models (OrderItem) become proper domain classes.

Module 5: inventoryWPF (Inventory — WPF)

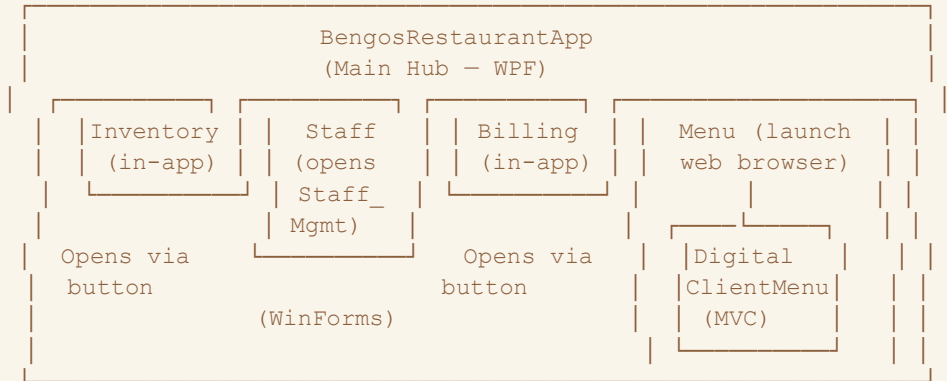
Project Path	inventoryWPF/inventoryWPF/
Technology	WPF (.NET 8.0-windows) — WinExe
Current Architecture	Mixed: Code-Behind + INotifyPropertyChanged Model
Recommended Architecture	MVVM (Model-View-ViewModel)
Description	Product inventory management with CRUD operations against SQL LocalDB. Features: DataGrid display, Add/Edit/Delete products, category filtering (ComboBox), inventory alerts/notifications. Uses ObservableCollection<Product> bound to a DataGrid.
Data Access	SQL LocalDB (Database_Products.mdf) via raw ADO.NET (SqlConnection, SqlCommand, SqlDataReader). System.Data.SqlClient 4.9.1.
Current Strength	Product class implements INotifyPropertyChanged — data binding infrastructure is partially in place. ObservableCollection enables automatic UI

	updates on collection changes.
Current Problem	SQL queries and CRUD logic are directly in MainWindow.xaml.cs event handlers (~240 lines). No separation between UI and data access. Connection string is hardcoded in code-behind.
MVVM Refactor	Extract InventoryViewModel with ObservableCollection<Product>, ICommand for Add/Edit/Delete/Filter. Create IProductRepository interface with ProductRepository implementation (moves ADO.NET out of the View). Bind DataGrid and form fields to ViewModel.

Module 6: Inventory (Inventory — WinForms, Legacy)

Project Path	Inventory/Inventory/
Technology	WinForms (.NET 8.0-windows) — WinExe
Current Architecture	Code-Behind (no formal pattern)
Recommended Architecture	Deprecated — consolidate into inventoryWPF
Description	Earlier/alternative inventory management application using WinForms. Single Form1 with standard WinForms CRUD against a database (connection string in App.config). Appears to be a prototype or alternate version of inventoryWPF.
Data Access	Database via App.config connection string (likely LocalDB).
Recommendation	This module duplicates inventoryWPF functionality. Recommended to archive or remove, consolidating all inventory management into the WPF version (module 5).

3. Module Dependency Map



Standalone modules (not launched from hub):

- inventoryWPF (WPF – separate .sln)
- Invetory (WinForms – legacy, standalone)

4. Master Summary Table

Module	Technology	Current Arch	Recommended	Data Access	Description
BengosRestaurantApp	WPF .NET 8	Code-Behind	MVVM	Text files, in-memory	Central hub launcher with 7 windows. Dashboard navigation to all sub-systems.
DigitalClientMenu	ASP.NET Core MVC 10	MVC ☒	MVC (keep)	EF Core + SQLite	Web digital menu. Customers browse dishes by category. Seed data.
Staff_Management	WinForms .NET 8	Code-Behind	MVP	Text files (users.txt, shifts.txt)	Staff login and shift CRUD. Role-based permissions (Admin vs staff).
Billing&Payments	WinForms .NET 8	Code-Behind	MVP	In-memory List	Order billing: qty edit, discount popup, cash/card payment.
inventoryWPF	WPF .NET 8	Code-Behind + INPC	MVVM	SQL LocalDB (ADO.NET)	Product inventory CRUD with DataGrid and category filtering.
Invetory	WinForms .NET 8	Code-Behind	Deprecated	DB (connection string)	Legacy WinForms inventory app. Duplicates module 5.